

机器学习 作业一

221900175 毛九弢

一. (30 points) 性能度量

学习器 $\mathcal{L}$ 在某个多分类任务数据集上的预测混淆矩阵如表 1 所示，请回答下列问题。本题的答案请以分式或者小数点后两位的形式给出，比如 $P=0.67$ 。

真实情况	预测结果		
	第 1 类	第 2 类	第 3 类
第 1 类	7	1	4
第 2 类	2	6	4
第 3 类	2	2	8

表 1: 学习器 $\mathcal{L}$ 在某个多分类任务数据集上的预测混淆矩阵

这里把预测结果写为矩阵形式方便表示：

$$M = \begin{bmatrix} 7 & 1 & 4 \\ 2 & 6 & 4 \\ 2 & 2 & 8 \end{bmatrix}$$

1. 该学习器的预测准确率是多少？ (5 points)

ACC:

$$\text{acc} = \frac{\text{tr}(M)}{\sum_{ij} m_{ij}} = \frac{21}{36} = \frac{7}{12} \approx 0.58 \tag{1}$$

2. 计算该学习器的微查准率 (micro-P)、宏查准率 (macro-P)、微查全率 (micro-R) 和宏查全率 (macro-R)。 (10 points)

首先，计算以 1、2、3 分别为正例的查准率和查全率。

$$P_1 = \frac{m_{11}}{\sum m_{i1}} = \frac{7}{11} \approx 0.64$$

$$R_1 = \frac{m_{11}}{\sum m_{1j}} = \frac{7}{12} \approx 0.58$$

$$P_2 = \frac{m_{22}}{\sum m_{i2}} = \frac{6}{9} = \frac{2}{3} \approx 0.67$$

$$R_2 = \frac{m_{22}}{\sum m_{2j}} = \frac{6}{12} = \frac{1}{2} = 0.50$$

$$P_3 = \frac{m_{33}}{\sum m_{i3}} = \frac{8}{16} = \frac{1}{2} = 0.50$$

$$R_3 = \frac{m_{33}}{\sum m_{3j}} = \frac{8}{12} = \frac{2}{3} \approx 0.67$$

◦ 微查准确率 **micro-P**

$$\mathbf{micro-P} = \frac{\overline{TP}}{\overline{TP} + \overline{FP}} = \frac{\sum_d TP_d}{\sum_d TP_d + \sum_d FP_d} = \frac{tr(M)}{\sum_d (\sum_i m_{id})} = \mathbf{acc} = \frac{7}{12} \approx 0.58(2)$$

◦ 宏查准确率 **macro-P** =

$$\mathbf{macro-P} = \frac{1}{n} \sum_{1 \leq d \leq n} P_d \approx 0.60 \quad (3)$$

◦ 微查全率 **micro-R**

$$\mathbf{micro-R} = \frac{\overline{TP}}{\overline{TP} + \overline{FN}} = \frac{\sum_d TP_d}{\sum_d TP_d + \sum_d FN_d} = \frac{tr(M)}{\sum_d (\sum_j m_{dj})} = \mathbf{acc} = \frac{7}{12} \approx 0.58(4)$$

◦ 宏查全率 **macro-R**

◦
$$\mathbf{macro-R} = \frac{1}{n} \sum_{1 \leq d \leq n} R_d \approx 0.58 \quad (5)$$

3. 计算该学习器的微 F1 (micro-F1) 和宏 F1 (macro-F1)。 (5 points)

微 **F1 micro-F1**

$$\mathbf{micro-F1} = \frac{2}{\frac{1}{\mathbf{micro-P}} + \frac{1}{\mathbf{micro-R}}} = \frac{7}{12} \approx 0.58 \quad (6)$$

宏 **F1 macro-F1**

$$\mathbf{macro-F1} = \frac{2}{\frac{1}{\mathbf{macro-P}} + \frac{1}{\mathbf{macro-R}}} \approx 0.59 \quad (7)$$

4. 在多分类中，每个样例只有一个标签。而在多标签分类中，每个样例可以有多个标签。如表 2 所示，一共有 3 个标签，样例 x_1 的标签是 1 和 3，学习器的预测是 1 和 2。请根据教材 2.3.2 查准率、查全率与 F1 章节的描述和表 2 的样例，计算学习器 $\mathcal{L}_1$ 的微查准率、微查全率、微 F1、宏查准率、宏查全率和宏 F1。[提示: 依然是利用各类的混淆矩阵计算微 F1 和宏 F1.] (10 points)

样例	x_1	x_2	x_3	x_4
标签	1,3	1,2	2,3	1,2,3
学习器预测	1,2	2,3	2,3	1,3

表 2: 学习器 $\mathcal{L}_1$ 的样例表

思考TP等含义为：

- TP i in label and prediction
- FN i in label but not in prediction
- FP i not in label but in prediction
- TN
 i not in label and not prediction

如是，得右方表格：

标签类\ 样本	1	2	3
x_1	TP	FP	FN
x_2	FN	TP	FP
x_3	TN	TP	TP
x_4	TP	FN	TP

3.1 计算微xx

- 微查准率 **micro-P** =

$$\frac{\overline{TP}}{\overline{TP} + \overline{FP}} = \frac{\sum TP}{\sum TP + \sum FP} = \frac{6}{6 + 2} = \frac{3}{4} = 0.75(8)$$

- 微查全率 **micro-R** =

$$\frac{\overline{TP}}{\overline{TP} + \overline{FN}} = \frac{\sum TP}{\sum TP + \sum FN} = \frac{6}{6 + 3} = \frac{2}{3} \approx 0.67(9)$$

- 微 **F1 micro-F1** =

$$\frac{2}{\frac{1}{\text{micro-P}} + \frac{1}{\text{micro-R}}} = \frac{12}{17} \approx 0.71(10)$$

TFPN\ 标签类	TP	FN	FP	TN
1	2	1	0	1
2	2	1	1	0
3	2	1	1	0
平均	2	1	2/3	1/3

3.2 计算宏xx

- 标签1：

TP = 2, FP = 0, FN = 1

$P_1 = 2 / (2 + 0) = 1$

标签2：

TP = 2, FP = 1, FN = 1

$P_2 = 2 / (2 + 1) = 2/3$

标签3：

TP = 2, FP = 1, FN = 1

$P_3 = 2 / (2 + 1) = 2/3$

$$R_1 = 2 / (2 + 1) = 2/3$$

$$R_2 = 2 / (2 + 1) = 2/3$$

$$R_3 = 2 / (2 + 1) = 2/3$$

- 宏查准率 **macro-P**

$$\text{macro-P} = \frac{1}{n} \sum_{1 \leq d \leq n} P_d = \frac{1 + 2/3 + 2/3}{3} = \frac{7}{9} \approx 0.78 \quad (11)$$

- 宏查全率 **macro-R**

$$\text{macro-R} = \frac{1}{n} \sum_{1 \leq d \leq n} R_d = \frac{2/3 + 2/3 + 2/3}{3} = \frac{2}{3} \approx 0.67 \quad (12)$$

- 宏 F1 **macro-F1**

$$\text{macro-F1} = \frac{2}{\frac{1}{\text{macro-P}} + \frac{1}{\text{macro-R}}} = \frac{2}{\frac{1}{7/9} + \frac{1}{2/3}} = \frac{28}{39} \approx 0.72 \quad (13)$$

Python code to check the answer

```

1 from sklearn.metrics import precision_score, recall_score
2
3 def cal_p_and_r(y_true, y_pred):
4     micro_precision = precision_score(y_true, y_pred, average='micro')
5     macro_precision = precision_score(y_true, y_pred, average='macro')
6     micro_recall = recall_score(y_true, y_pred, average='micro')
7     macro_recall = recall_score(y_true, y_pred, average='macro')
8     micro_f1 = 2*micro_precision*micro_recall/(micro_precision+micro_recall)
9     macro_f1 = 2*macro_precision*macro_recall/(macro_precision+macro_recall)
10
11     print(f"Micro Precision: {micro_precision}")
12     print(f"Macro Precision: {macro_precision}")
13     print(f"Micro Recall: {micro_recall}")
14     print(f"Macro Recall: {macro_recall}")
15     print(f"Micro F1: {micro_f1}")
16     print(f"Macro F1: {macro_f1}")
17
18 def task2_3():
19     M = [
20         [7,1,4],
21         [2,6,4],
22         [2,2,8]

```

```

23     ]
24     y_true = []
25     y_pred = []
26     for i,mi in enumerate(M):
27         for j, mij in enumerate(mi):
28             y_true+=[i]*mij
29             y_pred+=[j]*mij
30     cal_p_and_r(y_true, y_pred)
31
32 def task4():
33     y_true = [[1, 0, 1], [1, 1, 0], [0, 1, 1], [1, 1, 1]]
34     y_pred = [[1, 1, 0], [0, 1, 1], [0, 1, 1], [1, 0, 1]]
35     cal_p_and_r(y_true, y_pred)
36
37 if __name__=="__main__":
38     print("Task2-3:")
39     task2_3()
40     print("\nTask4:")
41     task4()
42
43     """output:
44     Task2-3:
45     Micro Precision: 0.5833333333333334
46     Macro Precision: 0.601010101010101
47     Micro Recall: 0.5833333333333334
48     Macro Recall: 0.5833333333333334
49     Micro F1: 0.5833333333333334
50     Macro F1: 0.592039800995025
51
52     Task4:
53     Task4:
54     Micro Precision: 0.75
55     Macro Precision: 0.7777777777777777
56     Micro Recall: 0.6666666666666666
57     Macro Recall: 0.6666666666666666
58     Micro F1: 0.7058823529411765
59     Macro F1: 0.7179487179487178
60     """

```

二. (30 points) 性能度量

假设数据集包含 10 个样例，其对应的真实标签和学习器的输出值 (从大到小排列) 如表 3 所示。该任务是一个二分类任务，标签 1 或 0 表示真实标签为正例或负例。学习器的输出值代表学习器对该样例是正例的置信度 (认为该样例是正例的概率)。

样例	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
标签	1	1	0	1	1	0	1	0	0	0
学习器输出值	0.92	0.75	0.62	0.55	0.49	0.4	0.31	0.28	0.2	0.1

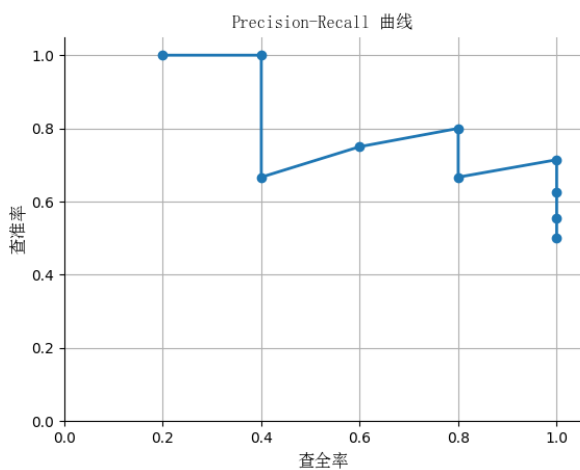
表 3: 样例表

Lb Th \ 样例	label	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
x_1	1	1 - TP									
x_2	1	0 - FN	1 - TP								
x_3	0	0 - TN		1 - FP							
x_4	1	0 - FN			1-TP						
x_5	1	0 - FN				1-TP					
x_6	0	0 - TN					1-FP				
x_7	1	0 - FN						1-TP			
x_8	0	0 - TN							1-FP		
x_9	0	0 - TN								1-FP	
x_{10}	0	0 - TN									1-FP
TP	/	1	2	2	3	4	4	5	5	5	5
FP	/	0	0	1	1	1	2	2	3	4	5
FN	/	4	3	3	2	1	1	0	0	0	0
TN	/	5	5	4	4	4	3	3	2	1	0

1. 计算P-R 曲线每一个端点的坐标并绘图 (8 points);

降序排序，按不同阈值进行计算

值坐标\ 样例	R	P	Co
x_1	1/5	1/1	(0.2, 1.0)
x_2	2/5	2/2	(0.4, 1.0)
x_3	2/5	2/3	(0.4, 0.67)
x_4	3/5	3/4	(0.6, 0.75)
x_5	4/5	4/5	(0.8, 0.8)
x_6	4/5	4/6	(0.8, 0.67)
x_7	5/5	5/7	(1.0, 0.71)
x_8	5/5	5/8	(1.0, 0.625)
x_9	5/5	5/9	(1.0, 0.56)
x_{10}	5/5	5/10	(1.0, 0.5)



```

1 import matplotlib.pyplot as plt
2 from matplotlib.font_manager
   import FontProperties
3
4 font =
   FontProperties(fname="C:/Window
   s/Fonts/simsun.ttc", size=12)
5
6 recall = [1 / 5, 2 / 5, 2 / 5,
   3 / 5, 4 / 5, 4 / 5, 5 / 5, 5
   / 5, 5 / 5, 5 / 5]
7 precision = [1 / 1, 2 / 2, 2 /
   3, 3 / 4, 4 / 5, 4 / 6, 5 / 7,
   5 / 8, 5 / 9, 5 / 10]
8
9 plt.figure()
10
11 plt.grid(True)
12 plt.plot(recall, precision, "-
   o", linewidth=2, markersize=6)
13
14 plt.xlabel("查全率",
   fontproperties=font)
15 plt.ylabel("查准率",
   fontproperties=font)
16
17 plt.title("Precision-Recall 曲
   线", fontproperties=font)
18 plt.xlim([0, 1.05])
19 plt.ylim([0, 1.05])
20
21 ax = plt.gca()
22 # 处理边框和保存图片省略
23 # ...

```

2. 计算ROC 曲线每一个端点的坐标并绘图 (8 points);

进行计算

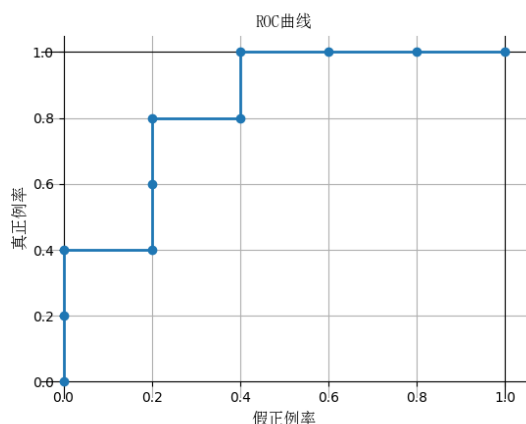
值坐标\ 样例	FPR	TRR	Co
x_1	0/5	1/5	(0, 0.2)
x_2	0/5	2/5	(0, 0.4)

```

1 import matplotlib.pyplot as plt
2 from matplotlib.font_manager
   import FontProperties
3
4 font =
   FontProperties(fname="C:/Windows/F

```

x_3	1/5	2/5	(0.2, 0.4)
x_4	1/5	3/5	(0.2, 0.6)
x_5	1/5	4/5	(0.2, 0.8)
x_6	2/5	4/5	(0.4, 0.8)
x_7	2/5	5/5	(0.4, 1.0)
x_8	3/5	5/5	(0.6, 1.0)
x_9	4/5	5/5	(0.8, 1.0)
x_{10}	5/5	5/5	(1.0, 1.0)



```

ons/simsun.ttc", size=12)
5
6 FPR = [0, 0 / 5, 0 / 5, 1 / 5, 1
  / 5, 1 / 5, 2 / 5, 2 / 5, 3 / 5, 4
  / 5, 5 / 5]
7 TRR = [0, 1 / 5, 2 / 5, 2 / 5, 3
  / 5, 4 / 5, 4 / 5, 5 / 5, 5 / 5, 5
  / 5, 5 / 5]
8
9 plt.figure()
10 plt.plot(FPR, TRR, "-o",
  linewidth=2, markersize=6,
  zorder=10)
11 plt.grid(True)
12
13 plt.xlabel("假正例率",
  fontproperties=font)
14 plt.ylabel("真正例率",
  fontproperties=font)
15 plt.title("ROC曲线",
  fontproperties=font)
16 plt.xlim([-0.05, 1.05])
17 plt.ylim([-0.05, 1.05])
18
19 ax = plt.gca()
20 # 处理边框和保存图片省略
21 # ...

```

3. 基于上一问，计算 AUC 的值 (4 points); AUC 值请以小数点后两位的形式给出。

计算折线下方的面积，得：

$$AUC = 0.84 \quad (14)$$

4. FPR95 是一个常见的性能度量指标，它指的是当真正例率 (true positive rate) 为 95% 时，假正例率 (false positive rate) 的数值。请问该指标越高学习性能越好还是越低性能越好，并且求解 FPR75 为多少。[提示: FPR75 和 FPR95 类似, FPR75 是真正例率为 75%。](10 points)

这个性能指标应该是 **越低越好**。

根据第2问的图像得：

$$FPR75 = 0.2 \quad (15)$$

三. (10 points) 规范化

Min-max 规范化和 z-score 规范化是两种常见的规范化方法。两种规范化方法分别如下面的公式所示:

$$x' = x'_{min} + \frac{x - x_{min}}{x_{max} - x_{min}} \times (x'_{max} - x'_{min}) \quad (1)$$

$$x' = \frac{x - \bar{x}}{\sigma_x} \quad (2)$$

其中 x 和 x' 分别是规范化前后的值, x_{max} 和 x_{min} 分别是规范化前的最大值和最小值, x'_{max} 和 x'_{min} 分别是规范化后的最大值和最小值, $\bar{x}$ 和 σ_x 是规范化前的均值和标准差。请分析二种规范化的优缺点。

本身并没有过于绝对的优缺点, 还是要看**使用场景**和**要求**

Min-max规范化

优点

- 保持了数据的**原始分布**。
- 可以**控制**规范化后数据的**范围** $[x'_{min}, x'_{max}]$ 。
- 适合一些需要**异常值敏感**的算法。
- 如果数据的**最小值和最大值**较为稳定, 就不需要重新规范化。

缺点

- **异常值敏感**, 极端大和极端小的值会导致归一化偏离。
- 如果数据的**最小值和最大值**经常发生变化, 需要重新规范化。

z-score规范化

这种方法将数据的均值规范化为0, 标准差规范化为1。

优点

- 对**异常值不敏感**, 有一定的**鲁棒性**, 因为它们不会严重影响均值和标准差的计算。
- 使得数据具有**标准正态分布**的性质, 均值为0, 标准差为1, 适用于假设数据近似正态分布的算法。

缺点

- 规范化后的**数据范围不固定**的。
- 每一次数据增多都要**重新规范化**。
- 如果原数据不是近似正态分布, **改变了数据的分布**。

四. (30 points) 线性回归

给定包含 m 个样例的数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$, 其中 $\mathbf{x}_i = (x_{i1}; x_{i2}; \dots; x_{id}) \in \mathbb{R}^d$, $y_i \in \mathbb{R}$ 为 $\mathbf{x}_i$ 的实数标记。线性回归模型要求该线性模型的预测结果和其所对应的标记之间的误差之和最小:

$$(\mathbf{w}^*, b^*) = \underset{(\mathbf{w}, b)}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^m (y_i - (\mathbf{w}^\top \mathbf{x}_i + b))^2 \quad (3)$$

即寻找一组权重 $(\mathbf{w}, b)$, 使其对 D 中示例预测的整体误差最小。定义 $y = [y_1; y_2; \dots; y_m] \in \mathbb{R}^m$ 且 $\mathbf{X} = [\mathbf{x}_1^\top; \mathbf{x}_2^\top; \dots; \mathbf{x}_m^\top] \in \mathbb{R}^{m \times d}$.

1. 请将线性回归的优化过程使用矩阵进行表示; (10 points)

$$\hat{\mathbf{w}} = (\mathbf{w}; b)$$

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} & 1 \\ x_{21} & x_{22} & \cdots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^\top & 1 \\ \mathbf{x}_2^\top & 1 \\ \vdots & \vdots \\ \mathbf{x}_m^\top & 1 \end{pmatrix}$$

$$\hat{\mathbf{w}}^* = \underset{\hat{\mathbf{w}}}{\operatorname{argmin}} (y - \mathbf{X}\hat{\mathbf{w}})^\top (y - \mathbf{X}\hat{\mathbf{w}}) \quad (16)$$

2. 使用矩阵形式给出权重 $\mathbf{w}^*$ 和偏移 b^* 最优解的表达式; (10 points)

令 $E_{\hat{\mathbf{w}}} = (y - \mathbf{X}\hat{\mathbf{w}})^\top (y - \mathbf{X}\hat{\mathbf{w}})$, 对 $\hat{\mathbf{w}}$ 求偏导得:

$$2\mathbf{X}^\top (\mathbf{X}\hat{\mathbf{w}} - y)$$

令其等于零

当 $\mathbf{X}^\top \mathbf{X}$ 正定时, 得到 $\mathbf{w}^*$ 和 b^* 的最优解表达式,

为:

$$(\mathbf{w}^*; b^*) = \hat{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top y \quad (17)$$

否则有多个解, 需要引入正则化项。

3. 针对波士顿房价预测数据集 (boston), 编程实现原始线性回归模型。请基于闭式解在训练集上构建模型, 计算测试集上的均方误差 (Mean Square Error, MSE)。请参考如下形式完成函数 `linear_regression` 和 `MSE` 的代码。除示例代码中使用到的 `sklearn` 库函数外, 不能使用其他的 `sklearn` 函数, 需要基于 `numpy` 实现线性回归模型闭式解以及 `MSE` 的计算. (10 points)

最终输出

MSE = 20.724023437339845

(18)

```
1 # from sklearn.datasets import load_boston
2 """
3 Traceback (most recent call last):
4   from sklearn.datasets import load_boston
5   File "D:\Program Files\python\python311\Lib\site-
     packages\sklearn\datasets\__init__.py", line 156, in __getattr__
6     raise ImportError(msg)
7 ImportError:
8 `load_boston` has been removed from scikit-learn since version 1.2.
9 """
```

由于 `load\_boston` 在 scikit-learn 1.2 之后弃用了. 所以, 我在官网找了

代码如下:

```
1 # from sklearn.datasets import load_boston
2 from sklearn.model_selection import train_test_split
3 import pandas as pd
4 import numpy as np
5 import os
6
7 # X,y = load_boston(return_X_y=True)
8
9 try:
10     if os.path.exists("boston_housing.csv"):
11         df = pd.read_csv("boston_housing.csv")
12         y = df["MEDV"] # 标签-房价
13         X = df.drop(["MEDV"], axis=1) # 去掉标签 (房价) 的数据子集
14         trainx, testx, trainy, testy = train_test_split(
15             X, y, test_size=0.33, random_state=42
16         )
17     else:
18         raise FileNotFoundError("boston_housing.csv not found")
19 except Exception as e:
20     print(e)
21     data_url = "http://lib.stat.cmu.edu/datasets/boston"
22     """ 长这样:
23     The Boston house-price data of Harrison, D. and Rubinfeld, D.L. 'Hedonic
24     prices and the demand for clean air', J. Environ. Economics & Management,
```

```

25     vol.5, 81-102, 1978.  Used in Belsley, Kuh & Welsch, 'Regression
    diagnostics
26     ...', Wiley, 1980.  N.B. Various transformations are used in the table on
27     pages 244-261 of the latter.
28
29     Variables in order:
30     CRIM      per capita crime rate by town
31     ZN        proportion of residential land zoned for lots over 25,000 sq.ft.
32     INDUS     proportion of non-retail business acres per town
33     CHAS      Charles River dummy variable (= 1 if tract bounds river; 0
    otherwise)
34     NOX       nitric oxides concentration (parts per 10 million)
35     RM        average number of rooms per dwelling
36     AGE       proportion of owner-occupied units built prior to 1940
37     DIS       weighted distances to five Boston employment centres
38     RAD       index of accessibility to radial highways
39     TAX       full-value property-tax rate per $10,000
40     PTRATIO   pupil-teacher ratio by town
41     B         1000(Bk - 0.63)^2 where Bk is the proportion of blacks by town
42     LSTAT     % lower status of the population
43     MEDV      Median value of owner-occupied homes in $1000's
44
45     0.00632  18.00   2.310  0  0.5380  6.5750  65.20  4.0900   1  296.0  15.30
46     396.90   4.98   24.00
47     0.02731   0.00   7.070  0  0.4690  6.4210  78.90  4.9671   2  242.0  17.80
48     396.90   9.14   21.60
49     """
50     raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
51     """
52     raw_df.values[:,2, :]: 选择 raw_df 中所有偶数行（从第0行开始，每隔一行取一次），并
    选择所有列。
53     raw_df.values[1::2, :2]: 选择 raw_df 中所有奇数行（从第1行开始，每隔一行取一次），
    并选择前两列。
54     np.hstack(...): 将上述两个数组水平堆叠（按列拼接）在一起，形成一个新的数组 x。
    这里也就是去掉目标房价MEDV后的内容
55     """
56
57     X = np.hstack([raw_df.values[:,2, :], raw_df.values[1::2, :2]])
58     # 选择 raw_df 中所有奇数行（从第1行开始，每隔一行取一次），并选择第3列（索引为2） 也
    就是目标房价MEDV
59     y = raw_df.values[1::2, 2]
60     trainx, testx, trainy, testy = train_test_split(
61         X, y, test_size=0.33, random_state=42
62     )
63     # 将特征矩阵和目标向量组合成一个 DataFrame
64     df = pd.DataFrame(
65         np.hstack([X, y.reshape(-1, 1)]),
66         columns=[

```

```

67         "CRIM",
68         "ZN",
69         "INDUS",
70         "CHAS",
71         "NOX",
72         "RM",
73         "AGE",
74         "DIS",
75         "RAD",
76         "TAX",
77         "PTRATIO",
78         "B",
79         "LSTAT",
80         "MEDV",
81     ],
82 )
83
84     # 保存为 CSV 文件
85     df.to_csv("boston_housing.csv", index=False)
86     print("boston_housing.csv saved")
87
88 def linear_regression(X_train, y_train):
89     """线性回归
90
91     Args:
92         X_train (np.ndarray): n X ( d + 1 )
93         y_train (np.ndarray): n X 1
94     返回: 权重矩阵W
95     """
96     # 使用方程计算 W
97     #  $W = (X^T X)^{-1} * X^T * y$ 
98     W = np.linalg.inv(X_train.T @ X_train) @ X_train.T @ y_train
99     return W
100
101 def MSE(X_train, y_train, X_test, y_test):
102     """调用linear_regression得到权重矩阵W后计算均方误差MSE
103
104     Args:
105         X_train (np.ndarray): n X ( d + 1 )
106         y_train (np.ndarray): n X 1
107         X_test (np.ndarray): n X ( d + 1 )
108         y_test (np.ndarray): n X 1
109     返回: 标量, 均方误差MSE的值
110     """
111     W = linear_regression(X_train, y_train)
112     y_pred = X_test @ W
113     MSE = np.mean((y_pred - y_test) ** 2)

```

```
114     return MSE
115
116 # 添加一列全1
117 trainx = np.hstack([trainx,np.ones((trainx.shape[0], 1))])
118 testx = np.hstack([testx, np.ones((testx.shape[0], 1))])
119 linear_regression_MSE = MSE(trainx, trainy, testx, testy)
120 print(linear_regression_MSE) # 输出: 20.724023437334946
121
```